Word2vec from Scratch with Python and NumPy

TL;DR - word2vec is awesome, it's also really simple. Learn how it works, and implement your own version.

Introduction

Since joining a tech startup back in 2016, my life has revolved around machine learning and natural language processing (NLP). Trying to extract faint signals from terabytes of streaming social media is the name of the game. Because of this, I'm constantly experimenting and implementing different NLP schemes; word2vec being among the simplest and coincidently yielding great predictive value. The underpinnings of word2vec are exceptionally simple and the math is borderline elegant. The whole system is deceptively simple, and provides exceptional results. This tutorial aims to teach the basics of word2vec while building a barebones implementation in Python using NumPy. Note that the final Python implementation will not be optimized for speed or memory usage, but instead for easy understanding.

The goal with word2vec and most NLP embedding schemes is to translate text into vectors so that they can then be processed using operations from linear algebra. Vectorizing text data allows us to then create predictive models that use these vectors

as input to then perform something useful. If you understand both forward and back propagation for plain vanella neural networks, you already understand 90% of word2vec.

It's Actually Really Simple (I promise!)

Word2vec is actually a collection of two different methods: continuous bag-of-words (CBOW) and skip-gram[1]. Given a word in a sentence, lets call it w(t) (also called the *center word* or *target word*), CBOW uses the *context* or surrounding words as input. For instance, if the context window *C* is set to C=5, then the input would be words at positions w(t-2), w(t-1), w(t+1), and w(t+2). Basically the two words before and after the center word w(t). Given this information, CBOW then tries to predict the target word.

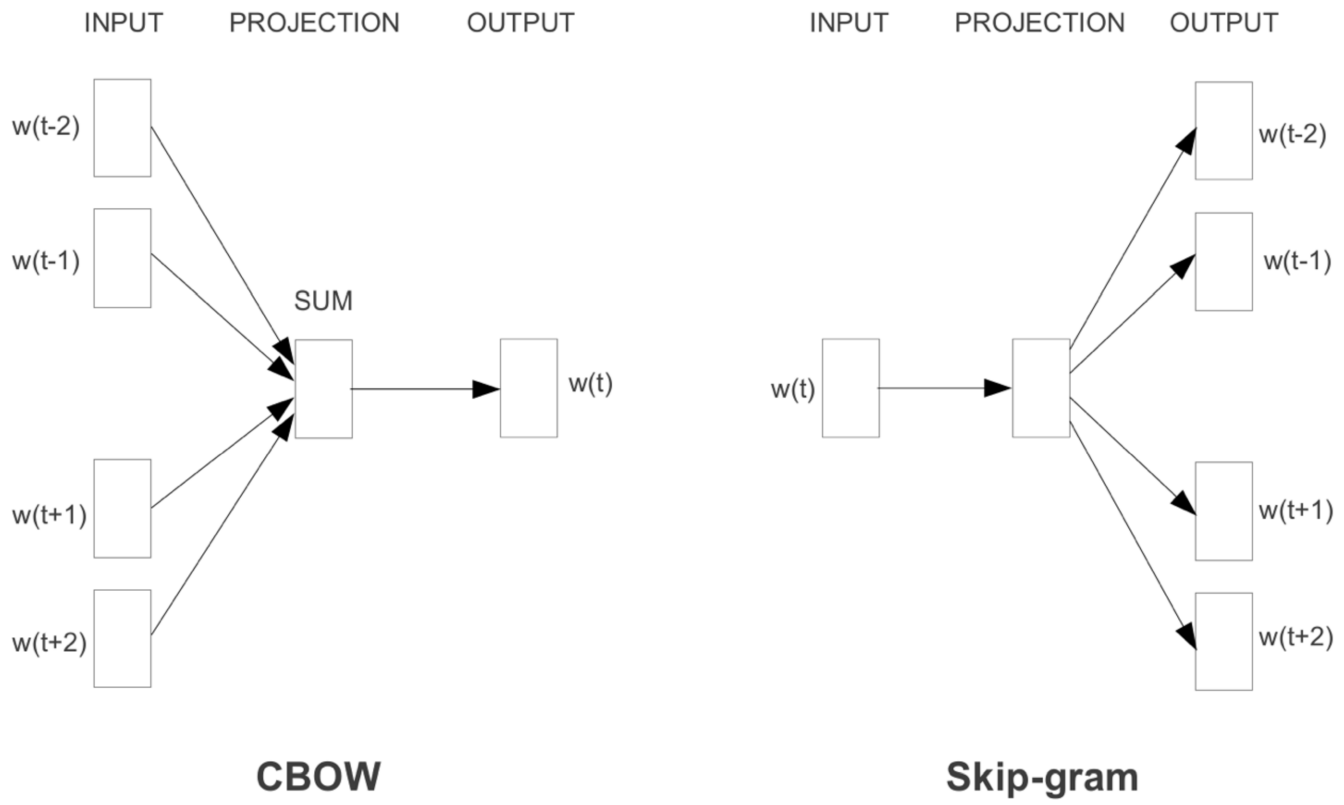

Figure 1: word2vec CBOW and skip-gram network architectures

The second method, skip-gram is the exact opposite. Instead of inputting the context words and predicting the center word, we feed in the center word and predict the context words. This means that w(t) becomes the input while w(t-2), w(t-1), w(t+1), and

w(t+2) are the ideal output. For for this post, we're only going to consider the skip-gram model since it has been shown to produce better word-embeddings than CBOW.

The concept of a center word surrounded by context words can be likened to a sliding window that travels across the text corpus. As an example, lets encode the following sentence: *"the quick brown fox jumps over the lazy dog"* using a window size of C=5 (two before, and two after the center word). As the context window slides across the sentence from left to right, it gets populated with the corresponding words. When the context window reaches the edges of the sentences, it simply drops the furthest window positions. Below is what this process looks like. Note that instead of w(t), w(t+1), etc., the center word has become x_k and the context words have become y_c.

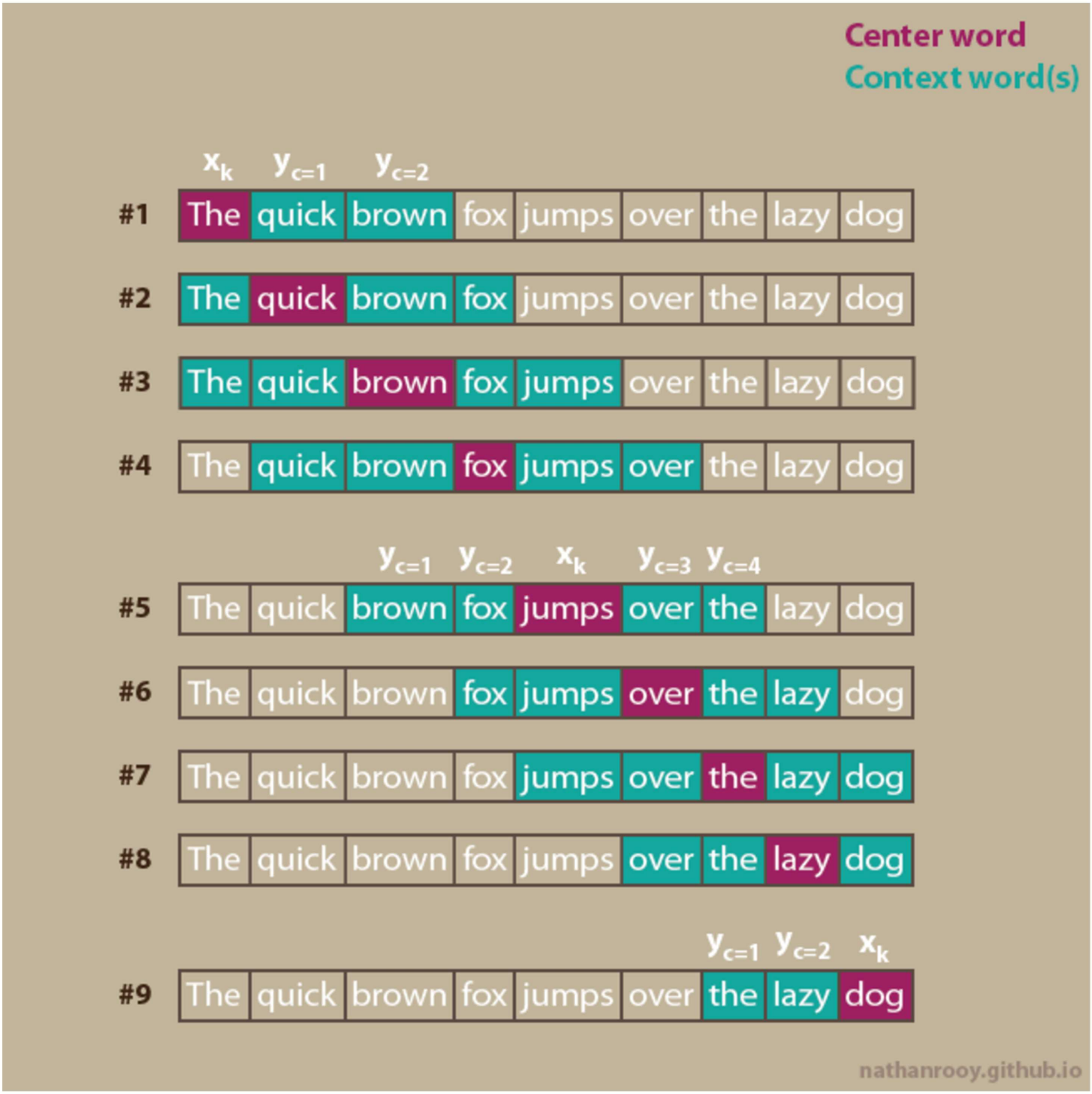

Figure 2: a sliding window example

Because we can't send text data directly through a matrix, we need to employ one-hot encoding. This means we have a vector of length *v* where v is the total number of unique words in the text corpus (or shorter if we want). Each word corresponds to a single position in this vector, so when embedding the word v_n, everywhere in vector v is zero except v_n which becomes a one. Below in Figure 3, a one-hot encoding of examples 1, 5,

and 9 from Figure 2 above.

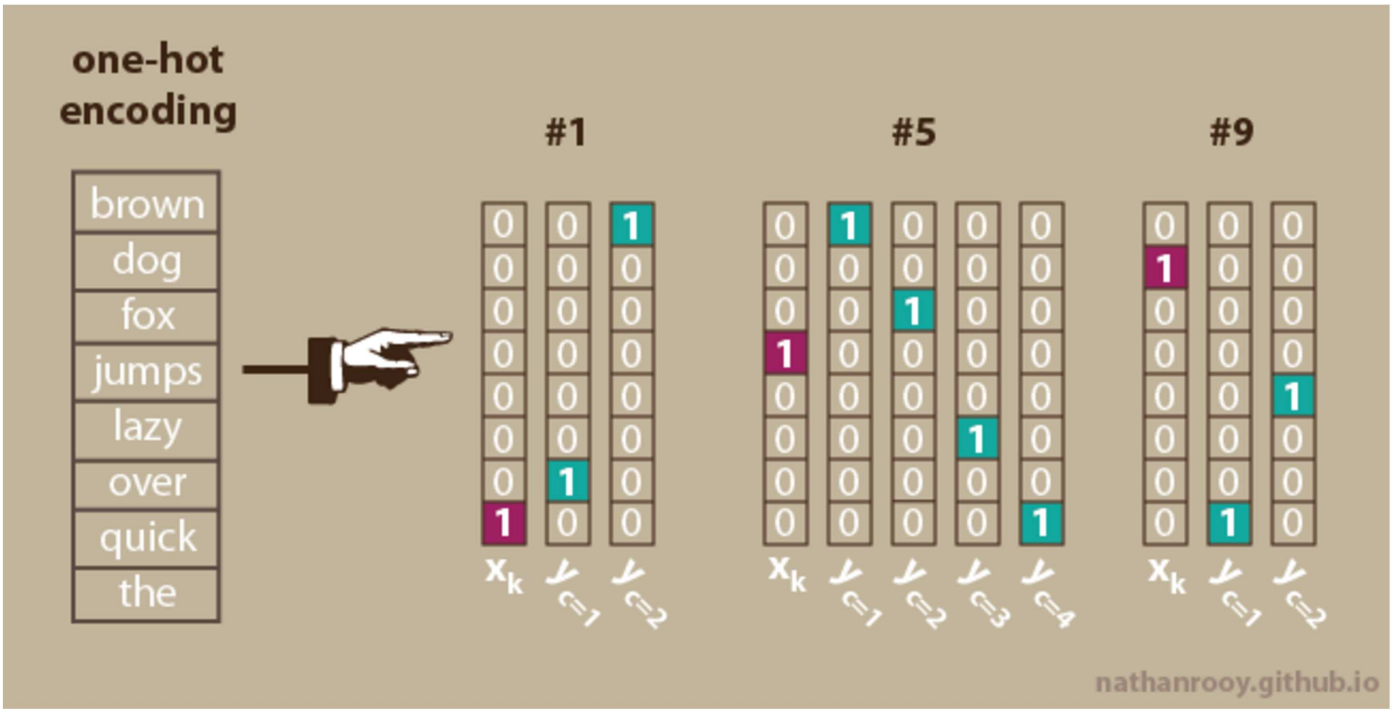

Figure 3: one-hot encoding

→ For future reference, the column vectors $y_{c=1}, ..., y_{c=C}$ are referred to as ***panels***.

So far, so good right? Now we need to feed this data into the network and train it. Most of the literature describing skip-gram uses the same graphic depicting the final layer of the model somehow having three or more matrices. I found this rather confusing while I was reading the white papers so I ended up digging through the original source code to get to the bottom of it. I figure there are other people like me, so I created another version of the architecture that I find a little easier to digest.

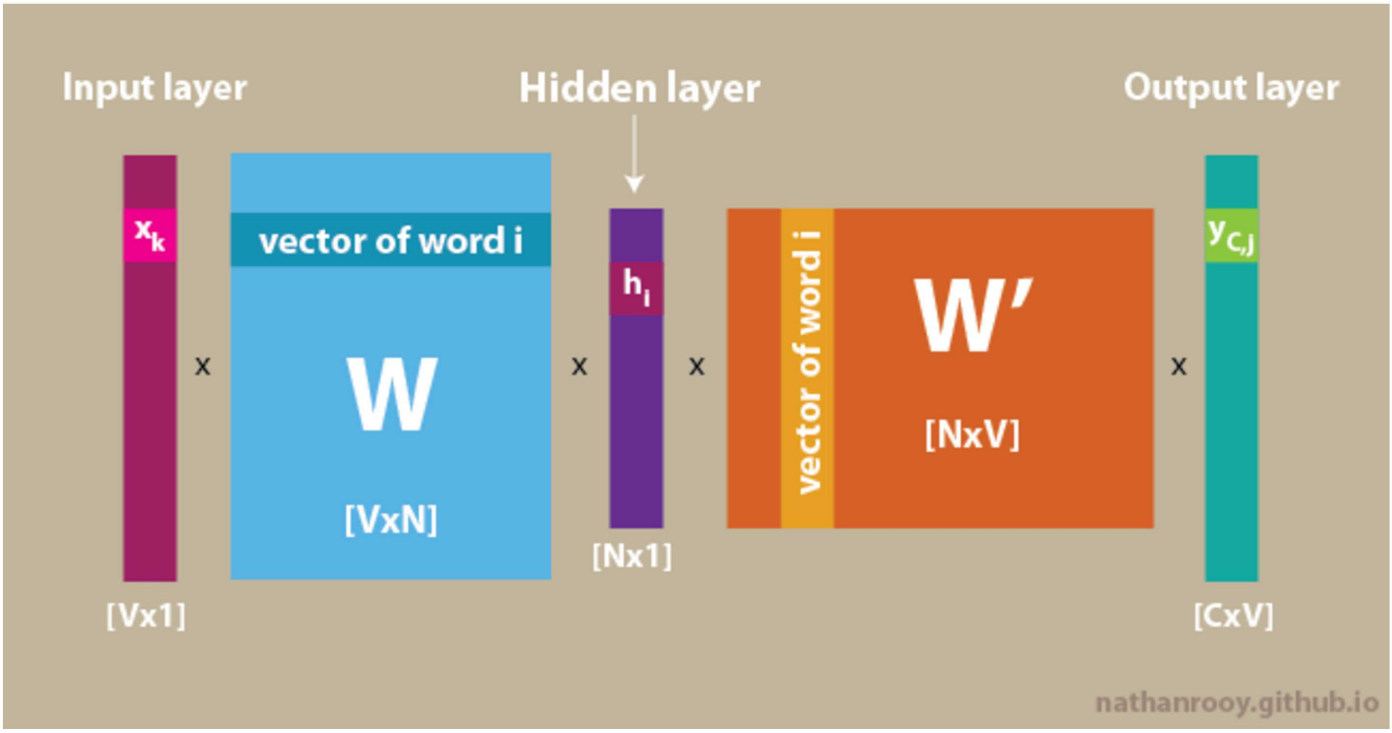

Figure 4: skip-gram network architecture

Forward Pass

Now that the text data has been encoded, lets proceed through a single forward pass of the network. Step one is getting to the hidden layer *h*.

$$h = x^T W$$

Since x is a one-hot encoded vector, h is simply the k^{th} row of matrix W.

$$h = W^T_{(k,:)} = v^T_{w_I}$$

Preceding forward, we need to calculate the values as they exit the network.

$$u_c = W'^T h = W'^T W^T x$$

Although u_c gets us through the network, we need to apply a softmax function. The softmax will compress each element of u_c to a range of [0,1] while also forcing the sum of u_c to equal one. This will help in computing network error and backpropagation later on.

$$
\begin{aligned}
y_c &= Softmax(u) \\
p\left(w_{c,j} = w_{O,c} | w_I\right) &= y_{c,j} = \frac{exp(u_{c,j})}{\sum_{j'=1}^{V} exp(u_{j'})}
\end{aligned}
$$

For each one-hot encoded center word x, matrix $W^{'}$ will output a certain set of values. This means that all the panels associated with center word x share the same input values, thus:

$$u_{c,j} = u_j = v_{w_j}^{'T} \cdot h \qquad c = 1, 2, ..., C$$ for

where $v_{w_j}^{'}$ is the output vector of the j-th word in the vocabulary, w_j. Again because we're dealing with one-hot encodings, this means $v_{w_j}^{'}$ is a column vector taken from the hidden → output matrix $W^{'}$. Refer to figure-4 for clarity.

In code, this will take the form of the following:

```
# FORWARD PASS
def forward_pass(self, x):
    h = np.dot(self.w1.T, x)
    u_c = np.dot(self.w2.T, h)
    y_c = self.softmax(u)
    return y_c, h, u
```

With the softmax calculation taking the form of:

```
# SOFTMAX ACTIVATION FUNCTION
```

```
    def softmax(self, x):
        e_x = np.exp(x - np.max(x))
        return e_x / e_x.sum(axis=0)
```

The forward pass is fairly simple and differs little from that of a standard, fully connected neural network.

Backpropagation & Training

Now to improve the weights within W and $W^{'}$ we're going to use stochastic gradient decent (SGD) to backpropagate the errors, which means we need to calculate the loss on the output layer.

$$
\begin{aligned}
E &= -\log \mathbb{P}(w_{O,1}, w_{O,2}, ..., w_{O,c} | w_I) \\
&= -\log \prod_{c=1}^{C} \frac{\exp(u_{c,j_c^*})}{\sum_{j^{'}=1}^{V} \exp(u_{j^{'}})} \\
&= -\sum_{c=1}^{C} u_{j_c^*} + C \cdot \log \sum_{j^{'}=1}^{V} \exp(u_{j^{'}})
\end{aligned}
$$

where j_c^* represents the index of the actual c-th output context word in the vocabulary.

Now that the loss has been calculated, we're going to employ the chain rule to distribute the error amongst the weight matrices W and $W^{'}$. First step is taking the derivative of E with respect to every element on every panel of the output layer $u_{c,j}$.

$$
\frac{\partial E}{\partial u_{c,j}} = y_{c,j} - t_{c,j} := e_{c,j}
$$

Where $t_{c,j}$ is the ground truth for that particular panel. To simplify the notation going forward, we'll define the following:

$$EI_j = \sum_{c=1}^{C} e_{c,j} = \sum_{c=1}^{C} \left(y_{c,j} - t_{c,j} \right) = \frac{\partial E}{\partial u_j} \tag{2}$$

The column vector EI_j represents the row-wise sum of the prediction errors across each context word panel for the current center word. Proceeding backwards, we need to take the derivative of E with respect to $W^{'}$ representing the output → hidden matrix.

$$\begin{aligned} \frac{\partial E}{\partial w_{ij}^{'}} &= \sum_{c=1}^{C} \frac{\partial E}{\partial u_{c,j}} \cdot \frac{\partial u_{c,j}}{\partial w_{i,j}^{'}} \\ &= \sum_{c=1}^{C} \left(y_{c,j} - t_{c,j} \right) \\ &= EI_j \cdot h_i \end{aligned}$$

Therefore, the gradient decent update equation for $W^{'}$ becomes:

$$w_{i,j}^{'(new)} = w_{i,j}^{'(old)} - \eta \cdot EI_j \cdot h_i$$

Note that η is the learning rate. Next, lets formulate the error update equation for the input → hidden layer W weights by deriving the error with respect to the hidden layer.

$$\begin{aligned} \frac{\partial E}{\partial h_i} &= \sum_{j=1}^{V} \frac{\partial E}{\partial u_j} \cdot \frac{\partial u_j}{\partial h_i} \\ &= \sum_{j=1}^{V} EI_j \cdot w_{ij}^{'} \end{aligned}$$

This allows us to then calculate the loss with respect to W.

$$\frac{\partial E}{\partial W_{ki}} = \frac{\partial E}{\partial h_i} \cdot \frac{\partial h_i}{\partial w_{ki}}$$
$$= \sum_{j=1}^{V} EI_j \cdot w'_{ij} \cdot x_k$$

and finally, we can formulate the gradient decent weight update equation for W.

$$w_{ij}^{(new)} = w_{ij}^{(old)} - \eta \cdot \sum_{j=1}^{V} EI_j \cdot w'_{ij} \cdot x_j$$

At this point, everything needed to train the network has been established and we just need to code it.

```
# TRAIN W2V model
def train(self, training_data):
    # INITIALIZE WEIGHT MATRICES
    self.w1 = np.random.uniform(-0.8, 0.8, (self.v_count, self.n))     # conte
    self.w2 = np.random.uniform(-0.8, 0.8, (self.n, self.v_count))     # embed

    # CYCLE THROUGH EACH EPOCH
    for i in range(0, self.epochs):

        self.loss = 0

        # CYCLE THROUGH EACH TRAINING SAMPLE
        for w_t, w_c in training_data:

            # FORWARD PASS
            y_pred, h, u = self.forward_pass(w_t)

            # CALCULATE ERROR
            EI = np.sum([np.subtract(y_pred, word) for word in w_c], axis=0)

            # BACKPROPAGATION
            self.backprop(EI, h, w_t)

            # CALCULATE LOSS
            self.loss += -np.sum([u[word.index(1)] for word in w_c]) + len(w_c

        print 'EPOCH:',i, 'LOSS:', self.loss
    pass
```

Where the backpropagation function is defined as:

```
# BACKPROPAGATION
def backprop(self, e, h, x):
    dl_dw2 = np.outer(h, e)
    dl_dw1 = np.outer(x, np.dot(self.w2, e.T))

    # UPDATE WEIGHTS
    self.w1 = self.w1 - (self.eta * dl_dw1)
    self.w2 = self.w2 - (self.eta * dl_dw2)
    pass
```

That's it! Only slightly more complicated than a simple neural network. To avoid posting redundant sections of code, you can find the completed word2vec model along with some additional features at this GitHub repo (link).

Results

As a simple sanity check, lets look at the network output given a few input words. This is the output after 5000 iterations.

Input word	brown	dog	fox	jumps	lazy	over	quick	the
fox	2.45e-01	4.34e-04	4.45e-04	2.53e-01	2.34e-05	2.53e-01	2.45e-01	7.62e-07
lazy	5.81e-05	3.32e-01	2.42e-04	1.11e-05	1.91e-04	3.33e-01	4.51e-04	3.33e-01
dog	1.85e-07	3.17e-04	1.31e-03	1.29e-04	4.98e-01	1.42e-05	4.86e-06	4.99e-01

These sample results show that the output probabilities for each center word are split fairly evenly between the correct context word. If we narrow in on the word ***lazy***, we can see that the probabilities for the words ***dog***, ***over***, and ***the*** are split fairly evenly at roughly 33.33% each. This is exactly what we want.

Improvements

Shortly after the initial release of word2vec, a second paper detailing several improvements was published.[2] Amongst these proposed improvements are:

Phrase Generation — This is the process in which commonly co-occuring words such as "san" and "francisco" become "san_francisco". The result of phrase generation is a cleaner, more useful, and user-friendly vocabulary list. Phrase generation is based on the following equation which utilizes the unigram and bigram vocabulary counts for the given corpus.

$$score(w_a, w_b) = \frac{count(w_a w_b) - \delta}{count(w_a) \times count(w_b)}$$

The numerator consists of the total number of times the bigram formed with words w_a and w_b appears in the corpus. This is then divided by the counts of w_a multiplied by w_b. The variable δ is referred to as the *discounting coefficient* and prevents the formation of word phrases consisting of very infrequent words.[2]

For longer word combinations such as "new york city", an iterative phrase generation approach can be used. As an example, on the initial pass "new" and "york" could be combined into "new_york", with a second pass combining "new_york" with "city" yielding the desired phrase "new_york_city". According to Mikolov et al. (2013), typically 2-4 passes with decreasing threshold values yielded the best results.

Subsampling — The purpose of subsampling is to counter the balance between frequent and rare words. No single word should represent a sizable portion of the corpus. To correct for this all words are discarded based on the following probability:

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

Where $f(w_i)$ is the frequency of word w_i and t is a user specified threshold. In Mikolov

et al. (2013), values for t were typically around 10^{-5}. As pointed out here, this probability calculation differs from the official word2vec C implementation. Below is the modified equation found in the C implementation:

$$P(w_i) = \left(\sqrt{\frac{z(w_i)}{0.001}} + 1\right) \times \frac{0.001}{z(w_i)}$$

Where $z(w_i)$ is the fraction of the corpus represented by the word w_i. The higher $P(w_i)$ is, the greather the chances are of keeping w_i.

Negative Sampling — Often referred to as just NEG, this is a modification to the backpropagation procedure in which only a small percentage of the errors are actually considered. For instance, using example #9 from figure 3 above, ***dog*** is the target word, while ***the*** and ***lazy*** are the context words. This means that in an ideal situation, the network will return a "0" for everything except for ***the*** and ***lazy*** which will be "1". In short, context words become "positive" while everything else becomes "negative". With negative sampling, we only concern ourselves with the positive words and only a small percentage of the negative words. This means that instead of backpropagating the errors from all 8 words in the vocabulary, we backpropagate the errors from the positive words ***the*** and ***lazy*** plus k negative words. Because the vocabulary size in this example is only 8, the advantage of negative sampling may not be immediately apparent. Now, imagine for a moment that you have a corpus that yielded billions of training samples, you've had to clip your vocabulary to 10,000 words and your embedding vector length is 300. With negative sampling we're updating a few positive words, and a few negative words (lets say $k = 10$) which translates to only 3,000 individual weights in W'. This represents 0.1% of the 3 million weight values we would otherwise be updating without negative sampling!

The probability that a negative word is chosen is determined using the following equation:

$$\log p(w|w_I) = \log\sigma(v'^T_w v_{w_I}) + \sum_{i=k}^{K} E_{w_i P_n(w)}[\log\sigma(-v'^T_w v_{w_I})]$$

As we can see, the primary difference between this and the standard stochastic gradient decent version is that now we include K observations. As for how large k should be, Mikolov et al. had this to say:

> Our experiments indicate that values of k in the range 5–20 are useful for small training datasets, while for large datasets the k can be as small as 2–5.
>
> - Mikolov et al. (2013)

Something to keep in mind however is that the official C implementation uses a slightly different formulation as seen below[3]:

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_{j=0}^{n}\left(f(w_j)^{3/4}\right)}$$

Note that even without negative sampling, only the weights for the target word in matrix W are updated.

Conclusion

Thanks for reading, I hope you found this useful! If anything still seems unclear, please let me know so I can improve this tutorial. For further reading, I highly suggest working through each of the references below.

1. Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean. *Efficient Estimation of Word Representations in Vector Space* (Submitted on 16 Jan 2013) ↩

2. Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, Jeffrey Dean. *Distributed Representations of Words and Phrases and their Compositionality* (Submitted on 16 Oct 2013) ↩ ↩

3. Chris McCormick. *Word2Vec Tutorial Part 2 - Negative Sampling* (January 11, 2017) ↩